Samraddhi Singhai 2024115015

Question 3

Section 1 — Global Header Validation

TC ID	Test Name	Input	Expected	Justification
H-01	Missing roll number	`X-User-ID: 1` only	401	Doc states every request needs `X-Roll-Number`. If missing, 401 is required. This is a gateway check that protects the entire API.
H-02	Non-integer roll number	`X-Roll-Number: ABC`	400	Doc states non-integer roll number returns 400. Ensures the header is validated as a number, not just present.
H-03	Missing user ID	`X-Roll-Number: 12345` only	400	Doc states user-scoped endpoints require `X-User-ID`. Missing it should be 400.
H-04	Non-integer user ID	`X-User-ID: abc`	400	Doc states invalid user ID returns 400. Validates type checking on the header.
H-05	Non-existent user ID	`X-User-ID: 999999`	400	Doc states user ID must match an existing user. Validates that the server looks the user up, not just checks the format.
H-06	Admin endpoint without user ID	`X-Roll-Number` only on `/admin/users`	200	Doc explicitly states admin endpoints do NOT require `X-User-ID`. Confirms the exemption works.

Section 2 — Admin Endpoints

TC ID	Test Name	Input	Expected	Justification
A-01	List all users	`GET /admin/users`	200, non-empty list	Baseline check. Confirms admin read works and seed data is present.
A-02	Get specific user	`GET /admin/users/1`	200, single user object	Validates specific-user lookup returns correct shape.
A-03	Get non-existent user	`GET /admin/users/999999`	404	Admin lookup of a missing user must not silently succeed.
A-04	Admin products includes inactive	`GET /admin/products`	List contains `is_active=false` items	Doc says admin sees all products including inactive. This distinguishes admin from the public endpoint.
A-05	Admin carts	`GET /admin/carts`	200, list	Confirms admin cart view is accessible.
A-06	Admin orders	`GET /admin/orders`	200, list	Confirms admin orders view is accessible.
A-07	Admin coupons includes expired	`GET /admin/coupons`	List contains expired coupons	Doc says admin sees all coupons including expired ones.
A-08	Admin tickets	`GET /admin/tickets`	200, list	Confirms admin tickets view is accessible.
A-09	Admin addresses	`GET /admin/addresses`	200, list	Confirms admin addresses view is accessible.

Section 3 — Profile

TC ID	Test Name	Input	Expected	Justification
P-01	Get profile	Valid headers	200 + `name`, `email`, `phone` fields	Baseline. Verifies the minimum response fields are returned.
P-02	Update valid profile	`name: "TestUser"`, `phone: "9876543210"`	200	Baseline update with a valid 8-character name and a 10-digit phone.
P-03	Name too short (1 char)	`name: "A"`	400	Doc: name must be ≥ 2 chars. "A" is the adjacent-below boundary.
P-04	Name too long (51 chars)	`name: "A"×51`	400	Doc: name must be ≤ 50 chars. 51 is one above the upper boundary.
P-05	Name exactly 2 chars	`name: "AB"`	200	Exact lower boundary must be accepted.
P-06	Name exactly 50 chars	`name: "A"×50`	200	Exact upper boundary must be accepted.
P-07	Phone 9 digits	`phone: "987654321"`	400	One digit short of the required 10.
P-08	Phone 11 digits	`phone: "98765432101"`	400	One digit over the required 10.
P-09	Phone with letters	`phone: "98765ABCDE"`	400	Doc requires exactly 10 digits. Letters must be rejected.
P-10	Missing name field	Body has only `phone`	400	Required field absent. Server must reject incomplete bodies.

P-11	Missing phone field	Body has only `name`	400	Required field absent. Server must reject incomplete bodies.

Section 4 — Addresses

TC ID	Test Name	Input	Expected	Justification
AD-01	Get addresses	Valid headers	200, list	Baseline read.
AD-02	Create valid address	All valid fields, `label: HOME`	200/201 + `address_id` in response	Doc says response must include full address object with assigned ID.
AD-03	Invalid label	`label: "WORK"`	400	Doc: only HOME, OFFICE, OTHER allowed.
AD-04	Street too short (4 chars)	`street: "AB12"`	400	Doc: street must be ≥ 5 chars. 4 is adjacent-below.
AD-05	Street exactly 5 chars	`street: "AB123"`	200/201	Exact lower boundary must be accepted.
AD-06	Street exactly 100 chars	`street: "A"×100`	200/201	Exact upper boundary must be accepted.
AD-07	Street 101 chars	`street: "A"×101`	400	One above the upper boundary must be rejected.
AD-08	City too short (1 char)	`city: "A"`	400	Doc: city must be ≥ 2 chars.
AD-09	Pincode 5 digits	`pincode: "12345"`	400	Doc: pincode must be exactly 6 digits. 5 is adjacent-below.

AD-10	Pincode 7 digits	`pincode: "1234567"`	400	One above the required 6.
AD-11	Pincode with letters	`pincode: "56000A"`	400	Must be all digits.
AD-12	Missing label	Body without `label`	400	Required field absent.
AD-13	Missing street	Body without `street`	400	Required field absent.
AD-14	Two default addresses	Set `is_default: true` twice	Only one default at a time	Doc: only one address can be default. Setting a new default must clear previous one.
AD-15	Update street	PUT new street value	200, response shows new street	Doc: update response must show new data, not old data.
AD-16	Delete non-existent address	`DELETE /addresses/999999`	404	Doc: deleting missing address returns 404.
AD-17	All valid label types	HOME, OFFICE, OTHER	200/201 for each	All three allowed labels must work individually.

Section 5 — Products

TC ID	Test Name	Input	Expected	Justification
PR-01	List products	`GET /products`	200, list	Baseline read.
PR-02	Only active in public list	Check `is_active` on every item	All items `is_active=true`	Doc: inactive products must never appear in the public listing.
PR-03	Get single product	`GET /products/1`	200, product object	Baseline single fetch.

PR-04	Get non-existent product	`GET /products/999999`	404	Doc: non-existent product ID returns 404.
PR-05	Price is exact	`price` field	Numeric, to 2 decimal places	Doc: price must be the exact stored value, not rounded or truncated.
PR-06	Filter by category	`?category=Fruits`	Only Fruits products	Verifies category filter correctness.
PR-07	Search by name	`?search=Apple`	Only Apple products	Verifies name search.
PR-08	Sort price ascending	`?sort=price_asc`	Prices ascending	Verifies sort order.
PR-09	Sort price descending	`?sort=price_desc`	Prices descending	Verifies sort order.
PR-10	Inactive hidden from public	Cross-check admin vs public	No inactive ID in public list	Confirms inactive-product filtering holds across both endpoints.

Section 6 — Cart

TC ID	Test Name	Input	Expected	Justification
C-01	Get cart	`GET /cart`	200, `items` + `total`	Baseline read.
C-02	Add valid item	`product_id: 1, quantity: 2`	200/201	Baseline add.
C-03	Quantity zero	`quantity: 0`	400	Doc: quantity must be ≥ 1. Zero is the adjacent-below boundary.

C-04	Negative quantity	`quantity: -1`	400	Negative quantities are meaningless inventory operations.
C-05	Non-existent product	`product_id: 999999`	404	Doc: non-existent product returns 404.
C-06	Over-stock quantity	`quantity: 999999`	400	Doc: quantity > stock must return 400.
C-07	Add same product twice	Add qty=2, then qty=3 for same product	Single row, qty=5	Doc: re-adding accumulates quantities rather than replacing.
C-08	Subtotal accuracy	Each item in cart	`subtotal = qty × unit_price`	Doc: each item subtotal must be calculated correctly.
C-09	Total accuracy	Cart with 2 products	`total = sum of subtotals`	Doc: every item must be counted; catches aggregation bugs.
C-10	Update quantity valid	`quantity: 4`	200	Baseline update.
C-11	Update to zero	`quantity: 0`	400	Same minimum rule applies to updates, not just adds.
C-12	Remove absent item	`product_id: 999999`	404	Doc: removing an item not in cart returns 404.
C-13	Clear cart	`DELETE /cart/clear`	200/204, then empty cart	Verifies clear empties the cart entirely.
C-14	Missing product_id	Body with only `quantity`	400	Required field absent.
C-15	Wrong type quantity	`quantity: "two"`	400	String where integer is required must be rejected.

Section 7 — Coupons

TC ID	Test Name	Input	Expected	Justification
CPN-01	Valid fixed coupon	`SAVE50` with cart ≥ 500	200	Baseline valid FIXED coupon.
CPN-02	Valid percent coupon	`PERCENT10` with cart ≥ 300	200	Baseline valid PERCENT coupon.
CPN-03	Expired coupon	`EXPIRED50` (expired 2026-02-23)	400	Doc: expired coupons must be rejected.
CPN-04	Cart below minimum	`SAVE50` with cart = 120 (min is 500)	400	Doc: cart total must meet coupon minimum.
CPN-05	Non-existent coupon	`FAKECOUPON999`	400/404	Non-existent code must be rejected.
CPN-06	Fixed discount amount	`SAVE50` on cart of 600	Discount exactly 50	Doc: FIXED discount equals the stated flat amount.
CPN-07	Percent discount cap	`PERCENT10` on cart of 1200 (10% = 120, cap = 100)	Discount ≤ 100	Doc: discount must not exceed the maximum cap.
CPN-08	Remove coupon	Apply then remove	Total restored to original	Removing coupon must fully undo the discount.
CPN-09	Missing coupon code	Body `{}`	400	`coupon_code` is required.
CPN-10	Second expired coupon	`FLASH25` (expired 2026-03-12)	400	Confirms expiry enforcement is not coupon-specific — it works generally.

Section 8 — Checkout

TC ID	Test Name	Input	Expected	Justification
CHK-01	COD valid cart	COD, total = 240	200/201	Baseline COD under 5000.
CHK-02	CARD payment status	`payment_method: CARD`	`payment_status: PAID`	Doc: CARD sets status to PAID immediately.
CHK-03	COD payment status	`payment_method: COD`	`payment_status: PENDING`	Doc: COD sets status to PENDING.
CHK-04	Empty cart	COD on empty cart	400	Doc: cannot checkout with no items.
CHK-05	Invalid method	`payment_method: CRYPTO`	400	Doc: only COD, WALLET, CARD are valid.
CHK-06	COD over 5000	COD, qty=42, total=5040	400	Doc: COD not allowed above 5000.
CHK-07	GST is 5%	Check `gst_amount` in response	`gst_amount = subtotal × 0.05`	Doc: GST is exactly 5%, applied once.
CHK-08	Missing payment method	Body `{}`	400	`payment_method` is required.
CHK-09	WALLET payment status	`payment_method: WALLET`	`payment_status: PENDING`	Doc: WALLET (like COD) starts as PENDING.

Section 9 — Wallet

TC ID	Test Name	Input	Expected	Justification

W-01	Get balance	`GET /wallet`	200, numeric balance	Baseline read.
W-02	Add valid amount	`amount: 100`	Balance increases by exactly 100	Doc: exact amount is added, no more or less.
W-03	Add zero	`amount: 0`	400	Doc: amount must be > 0.
W-04	Add negative	`amount: -50`	400	Negative amounts are nonsensical.
W-05	Add over 100000	`amount: 100001`	400	Doc: maximum is 100000. One above.
W-06	Add exactly 100000	`amount: 100000`	200	Exact upper boundary must be accepted.
W-07	Pay valid amount	Pay 100, balance > 100	Balance decreases by exactly 100	Doc: exact amount deducted, no extra.
W-08	Pay over balance	`amount: current_balance + 10000`	400	Doc: insufficient balance returns 400.
W-09	Pay zero	`amount: 0`	400	Same > 0 rule applies to pay.
W-10	Missing amount field	Body `{}`	400	`amount` is required.
W-11	String amount	`amount: "hundred"`	400	String where float is required must be rejected.

Section 10 — Loyalty Points

TC ID	Test Name	Input	Expected	Justification

L-01	Get points	`GET /loyalty`	200, integer `points`	Baseline read.
L-02	Redeem valid points	`points: 1` (if available)	200	Baseline redeem of minimum valid amount.
L-03	Redeem over limit	`points: 9999999`	400	Doc: must have enough points.
L-04	Redeem zero	`points: 0`	400	Doc: minimum is 1. Zero is adjacent-below.
L-05	Redeem negative	`points: -10`	400	Negative points make no sense.

Section 11 — Orders

TC ID	Test Name	Input	Expected	Justification
O-01	List orders	`GET /orders`	200, list	Baseline read.
O-02	Get single order	`GET /orders/{valid_id}`	200, order object	Baseline single fetch.
O-03	Get non-existent order	`GET /orders/999999`	404	Non-existent order.
O-04	Cancel valid order	POST cancel on PLACED order	200	Doc: PLACED orders can be cancelled.
O-05	Cancel non-existent	POST cancel on ID 999999	404	Doc: non-existent order returns 404.
O-06	Cancel delivered order	POST cancel on DELIVERED order	400	Doc: delivered orders cannot be cancelled.

O-07	Cancel restores stock	Cancel order, check stock	Stock restored to pre-order level	Doc: cancelled items must be returned to product stock.
O-08	Invoice fields present	`GET /orders/{id}/invoice`	`subtotal`, `gst_amount`, `total` all present	Doc: invoice must include all three fields.
O-09	Invoice total accurate	Compare invoice `total` to order `total_amount`	Must match exactly	Doc: invoice total must equal the actual order total.

Section 12 — Reviews

TC ID	Test Name	Input	Expected	Justification
R-01	List reviews	`GET /products/1/reviews`	200, list	Baseline read.
R-02	Post valid review	`rating: 5, comment: "Excellent!"`	200/201	Baseline create.
R-03	Rating exactly 1	`rating: 1`	200/201	Exact lower boundary must be accepted.
R-04	Rating exactly 5	`rating: 5`	200/201	Exact upper boundary must be accepted.
R-05	Rating 0	`rating: 0`	400	Doc: rating must be 1–5. Zero is adjacent-below the minimum.
R-06	Rating 6	`rating: 6`	400	Doc: rating must be 1–5. Six is adjacent-above the maximum.
R-07	Negative rating	`rating: -1`	400	Negative rating is invalid.

R-08	Empty comment	`comment: ""`	400	Doc: comment must be ≥ 1 char.
R-09	Comment 201 chars	`comment: "A"×201`	400	Doc: comment must be ≤ 200 chars. One above upper boundary.
R-10	Comment 200 chars	`comment: "A"×200`	200/201	Exact upper boundary must be accepted.
R-11	Non-existent product	`POST /products/999999/reviews`	404	Cannot review a product that doesn't exist.
R-12	Average is decimal	Post ratings 3 and 4	`average_rating: 3.5` (not 3)	Doc: average must be proper decimal, not integer-truncated.
R-13	No reviews → average 0	Product with no reviews	`average_rating: 0`	Doc: explicit zero when no reviews exist.
R-14	Missing rating field	Body with only `comment`	400	Rating is required.

Section 13 — Support Tickets

TC ID	Test Name	Input	Expected	Justification
ST-01	Create valid ticket	Valid subject + message	200/201	Baseline create.
ST-02	New ticket status	Valid ticket creation	`status: OPEN`	Doc: all new tickets must start with status OPEN.
ST-03	Subject too short	`subject: "Hi"` (2 chars)	400	Doc: subject must be ≥ 5 chars.
ST-04	Subject exactly 5 chars	`subject: "Hello"`	200/201	Exact lower boundary must be accepted.

ST-05	Subject 101 chars	`subject: "A"×101`	400	Doc: subject must be ≤ 100 chars. One above.
ST-06	Subject 100 chars	`subject: "A"×100`	200/201	Exact upper boundary must be accepted.
ST-07	Empty message	`message: ""`	400	Doc: message must be ≥ 1 char.
ST-08	Message 501 chars	`message: "A"×501`	400	Doc: message must be ≤ 500 chars. One above.
ST-09	Message 500 chars	`message: "A"×500`	200/201	Exact upper boundary must be accepted.
ST-10	Message stored exactly	Message with special chars	Retrieved message matches exactly	Doc: full message must be saved exactly as written.
ST-11	List tickets	`GET /support/tickets`	200, list	Baseline read.
ST-12	OPEN → IN_PROGRESS	PUT `status: IN_PROGRESS`	200	Doc: valid forward transition.
ST-13	IN_PROGRESS → CLOSED	PUT `status: CLOSED`	200	Doc: valid forward transition.
ST-14	OPEN → CLOSED (skip)	PUT `status: CLOSED` on OPEN ticket	400	Doc: must go through IN_PROGRESS first, cannot skip.
ST-15	CLOSED → OPEN (reverse)	PUT `status: OPEN` on CLOSED ticket	400	Doc: status only moves forward.
ST-16	Invalid status value	PUT `status: "PENDING"`	400	PENDING is not a valid ticket status.
ST-17	Missing subject	Body without `subject`	400	Subject is required.

ST-18	Missing message	Body without `message`	400	Message is required.

Part 2 — Bug Report

28 bugs found from the test run. Each is documented below with the exact payload sent, the expected behavior from the API documentation, and the actual observed behavior.

Bug 1 — Non-existent User ID returns 404 instead of 400

Endpoint: `GET /api/v1/profile` **Severity:** Low

Request:

```
Method:  GET
URL:     http://localhost:8080/api/v1/profile
Headers: X-Roll-Number: 12345
         X-User-ID: 999999
Body:    (none)
```

Expected: `400 Bad Request` The documentation states: *"All user-scoped endpoints require an X-User-ID header containing a positive integer matching an existing user. If it is missing or invalid, the server returns a 400 error."* A non-existent user ID is an invalid user ID.

Actual: `404 Not Found` The server returns 404 instead of 400 when the user does not exist.

Bug 2 — Phone number containing letters is accepted

Endpoint: `PUT /api/v1/profile` **Severity:** High

Request:

```
Method:  PUT
URL:     http://localhost:8080/api/v1/profile
```

Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: { "name": "Valid Name", "phone": "98765ABCDE" }

Expected: `400 Bad Request` The documentation states: *"The phone number must be exactly 10 digits."* The value `98765ABCDE` contains letters and must be rejected.

Actual: `200 OK` The server accepted a phone number with alphabetic characters. This allows corrupt data to be stored in the database.

Bug 3 — Valid address creation is rejected (returns 400)

Endpoint: `POST /api/v1/addresses` **Severity:** Critical

Request:

Method: POST
URL: http://localhost:8080/api/v1/addresses
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: {
"label": "HOME",
"street": "123 MG Road",
"city": "Bangalore",
"pincode": "560001",
"is_default": false
}

Expected: `200 OK` or `201 Created` with the created address object including `address_id` All fields are valid per the documentation rules.

Actual: `400 Bad Request` The server rejects a completely valid address payload. This makes the address creation endpoint entirely unusable. All subsequent address tests that rely on creation also fail as a cascade.

Bug 4 — Street with exactly 5 characters is rejected

Endpoint: `POST /api/v1/addresses` **Severity:** High

Request:

Method: POST
URL: http://localhost:8080/api/v1/addresses
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: { "label": "HOME", "street": "AB123", "city": "Bangalore",
"pincode": "560001", "is_default": false }

Expected: `200/201` The documentation states: *"The street must be between 5 and 100 characters."* A street of exactly 5 characters is the lower boundary and must be accepted.

Actual: `400 Bad Request` The server rejects a 5-character street, suggesting the actual minimum enforced is greater than 5, or the entire address endpoint has a broader issue (see Bug 3).

Bug 5 — Street with exactly 100 characters is rejected

Endpoint: `POST /api/v1/addresses` **Severity:** High

Request:

Method: POST
URL: http://localhost:8080/api/v1/addresses
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: { "label": "HOME", "street": "A"×100, "city": "Bangalore",
"pincode": "560001", "is_default": false }

Expected: `200/201` A street of exactly 100 characters is the upper boundary and must be accepted per the documentation.

Actual: `400 Bad Request` The server rejects a 100-character street, suggesting the maximum enforced is less than 100.

Bug 6 — Pincode with 5 digits is accepted (too short)

Endpoint: `POST /api/v1/addresses` **Severity:** High

Request:

Method: POST
URL: http://localhost:8080/api/v1/addresses

Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: { "label": "HOME", "street": "123 MG Road", "city": "Bangalore",
"pincode": "12345", "is_default": false }

Expected: `400 Bad Request` The documentation states: *"The pincode must be exactly 6 digits."* A 5-digit pincode is one below the required length and must be rejected.

Actual: `200 OK` The server accepts a 5-digit pincode. Pincode length validation is not enforced.

Bug 7 — Pincode with 7 digits is accepted (too long)

Endpoint: `POST /api/v1/addresses` **Severity:** High

Request:

Method: POST
URL: http://localhost:8080/api/v1/addresses
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: { "label": "HOME", "street": "123 MG Road", "city": "Bangalore",
"pincode": "1234567", "is_default": false }

Expected: `400 Bad Request` A 7-digit pincode exceeds the required exactly-6 rule and must be rejected.

Actual: `200 OK` The server accepts a 7-digit pincode. Pincode length validation is entirely absent.

Bug 8 — Cart accepts quantity of 0

Endpoint: `POST /api/v1/cart/add` **Severity:** High

Request:

Method: POST
URL: http://localhost:8080/api/v1/cart/add
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: { "product_id": 1, "quantity": 0 }

Expected: `400 Bad Request` The documentation states: *"When adding an item, the quantity must be at least 1. Sending 0 or a negative number must be rejected with a 400 error."*

Actual: `200 OK` The server silently accepts a quantity of 0. Adding zero items corrupts cart state.

Bug 9 — Cart accepts negative quantity

Endpoint: `POST /api/v1/cart/add` **Severity:** High

Request:

```
Method:  POST
URL:     http://localhost:8080/api/v1/cart/add
Headers: X-Roll-Number: 12345
         X-User-ID: 25
Body:    { "product_id": 1, "quantity": -1 }
```

Expected: `400 Bad Request` The documentation explicitly states: *"Sending 0 or a negative number must be rejected with a 400 error."*

Actual: `200 OK` The server accepts negative quantities. This is a critical inventory logic bug — a negative quantity would reduce cart totals below zero, which the subsequent total tests confirmed (totals of -16 and -142 were observed).

Bug 10 — Cart subtotal is calculated incorrectly

Endpoint: `GET /api/v1/cart` **Severity:** Critical

Request:

```
Method:  GET
URL:     http://localhost:8080/api/v1/cart
Headers: X-Roll-Number: 12345
         X-User-ID: 25
Body:    (none)
         [Cart state: product_id=1 at price=120, quantity=3]
```

Expected: `subtotal: 360` The subtotal for each item must equal `quantity × unit_price`. With product 1 at price 120 and quantity 3, the subtotal must be 360.

Actual: `subtotal: 104` The returned subtotal was 104, which does not correspond to any arithmetic on the correct inputs. The subtotal calculation is fundamentally broken.

Bug 11 — Cart total does not equal the sum of item subtotals

Endpoint: `GET /api/v1/cart` **Severity:** Critical

Request:

```
Method:  GET
URL:     http://localhost:8080/api/v1/cart
Headers: X-Roll-Number: 12345
         X-User-ID: 25
Body:    (none)
         [Cart state: product_id=1 qty=2, product_id=2 qty=1]
```

Expected: `total` equals the sum of all item subtotals The documentation states: *"The cart total must be the sum of all item subtotals. Every item must be counted, including the last one."*

Actual: `total: -16`, sum of subtotals: `-142` Both values are negative (caused by Bug 9 allowing negative quantities) and neither matches the other. The total aggregation is also independently incorrect.

Bug 12 — Missing product_id in cart add returns 404 instead of 400

Endpoint: `POST /api/v1/cart/add` **Severity:** Low

Request:

```
Method:  POST
URL:     http://localhost:8080/api/v1/cart/add
Headers: X-Roll-Number: 12345
         X-User-ID: 25
Body:    { "quantity": 1 }
```

Expected: `400 Bad Request` Missing a required field like `product_id` should return 400.

Actual: `404 Not Found` The server interprets the missing `product_id` as a lookup for a non-existent product and returns 404. The correct response for a missing required field is 400.

Bug 13 — Coupon applies when cart is below the minimum value

Endpoint: `POST /api/v1/coupon/apply` **Severity:** High

Request:

```
Method:  POST
URL:     http://localhost:8080/api/v1/coupon/apply
Headers: X-Roll-Number: 12345
         X-User-ID: 25
Body:    { "coupon_code": "SAVE50" }
         [Cart state: product_id=1, quantity=1, cart total=120]
         [SAVE50 minimum cart value = 500]
```

Expected: `400 Bad Request` The documentation states: *"The cart total must meet the coupon's minimum cart value."* SAVE50 requires a minimum of 500 and a cart of 120 must be rejected.

Actual: `200 OK` The coupon was accepted despite the cart total being far below the minimum. The minimum cart value check is not enforced.

Bug 14 — Fixed coupon discount is not applied (discount is 0)

Endpoint: `POST /api/v1/coupon/apply` **Severity:** Critical

Request:

```
Method:  POST
URL:     http://localhost:8080/api/v1/coupon/apply
Headers: X-Roll-Number: 12345
         X-User-ID: 25
Body:    { "coupon_code": "SAVE50" }
         [Cart state: total set to ≥ 600]
```

Expected: Cart total decreases by exactly 50 SAVE50 is a FIXED coupon with a discount value of 50. Applying it to a cart of 600 must reduce the total to 550.

Actual: Discount = 0 The cart total did not change at all after applying the coupon. The discount calculation for FIXED coupons is not being applied.

Bug 15 — COD checkout allowed when total exceeds 5000

Endpoint: `POST /api/v1/checkout` **Severity:** High

Request:

```
Method:  POST
URL:     http://localhost:8080/api/v1/checkout
Headers: X-Roll-Number: 12345
         X-User-ID: 25
Body:    { "payment_method": "COD" }
         [Cart state: product_id=1, quantity=42, total=5040]
```

Expected: `400 Bad Request` The documentation states: *"COD is not allowed if the order total is more than 5000. If someone tries, the server returns a 400 error."*

Actual: `200 OK` The server accepted a COD order with a total of 5040, which exceeds the 5000 limit. The COD limit check is not enforced.

Bug 16 — Wallet deducts more than the requested amount

Endpoint: `POST /api/v1/wallet/pay` **Severity:** High

Request:

```
Method:  POST
URL:     http://localhost:8080/api/v1/wallet/pay
Headers: X-Roll-Number: 12345
         X-User-ID: 25
Body:    { "amount": 100 }
         [Wallet balance before: 101614.38]
```

Expected: Balance after = 101514.38 (exactly 100 deducted) The documentation states: *"When paying from the wallet, the exact amount requested is deducted from the balance. No extra amount is taken."*

Actual: Balance after = 101514.18, deducted = 100.2 The server deducted 100.20 instead of 100.00. An extra 0.20 was taken from the user's wallet. This is a floating-point precision bug in the payment deduction logic.

Bug 17 — Cancelling a DELIVERED order returns 404 instead of 400

Endpoint: `POST /api/v1/orders/{order_id}/cancel` **Severity:** Medium

Request:

```
Method:  POST
URL:     http://localhost:8080/api/v1/orders/{delivered_order_id}/cancel
Headers: X-Roll-Number: 12345
         X-User-ID: 25
Body:    {}
```

Expected: `400 Bad Request` The documentation states: *"A delivered order cannot be cancelled. If someone tries, the server returns a 400 error."*

Actual: `404 Not Found` The server returns 404 instead of 400 when attempting to cancel a delivered order. The likely cause is that the server only allows users to cancel orders belonging to them, and the DELIVERED orders in the seed data belong to other users. However, the correct documented response for "cannot cancel a delivered order" is 400.

Bug 18 — Cancelling an order does not restore product stock

Endpoint: `POST /api/v1/orders/{order_id}/cancel` **Severity:** Critical

Request:

```
Method:  POST
URL:     http://localhost:8080/api/v1/orders/{order_id}/cancel
Headers: X-Roll-Number: 12345
```

```
        X-User-ID: 25
Body:   {}
        [Order contained product_id=1, quantity=2]
        [Stock before order: 139]
```

Expected: Stock of product 1 returns to 139 after cancellation The documentation states: *"When an order is cancelled, all the items in that order are added back to the product stock."*

Actual: Stock after cancellation = 137 Stock decreased by 2 instead of being restored. The order cancellation is deducting additional stock instead of restoring it, making the stock bug compound with every cancellation.

Bug 19 — Invoice response uses `total_amount` instead of `total`

Endpoint: `GET /api/v1/orders/{order_id}/invoice` **Severity:** Medium

Request:

```
Method:  GET
URL:     http://localhost:8080/api/v1/orders/{order_id}/invoice
Headers: X-Roll-Number: 12345
         X-User-ID: 25
Body:    (none)
```

Expected: Response contains `subtotal`, `gst_amount`, and `total` The documentation states the invoice must show all three fields. Test assertion: `assert "total" in invoice`.

Actual Response Body:

```
{
  "gst_amount": 12,
  "order_id": "3010",
  "subtotal": 240,
  "total_amount": 252
}
```

The field is named `total_amount` in the invoice response but the documentation and test expect `total`. The field `total` is absent. This is a field-naming inconsistency.

Bug 20 — Invoice total field mismatch prevents correct validation

Endpoint: `GET /api/v1/orders/{order_id}/invoice` **Severity:** Medium

Request:

Method: GET
URL: http://localhost:8080/api/v1/orders/{order_id}/invoice
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: (none)

Expected: `invoice["total"]` equals `order["total_amount"]` (252) The invoice total must match the order total exactly.

Actual: `invoice.get("total", 0)` returns 0 because the field is named `total_amount` not `total` (see Bug 19). The comparison fails as `0 != 252`.

Bug 21 — Review rating of 0 is accepted

Endpoint: `POST /api/v1/products/{product_id}/reviews` **Severity:** High

Request:

Method: POST
URL: http://localhost:8080/api/v1/products/1/reviews
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: { "rating": 0, "comment": "Nothing" }

Expected: `400 Bad Request` The documentation states: *"A review rating must be between 1 and 5."* Zero is below the minimum.

Actual: `200 OK` The server accepted a rating of 0. Rating validation is not enforced at the lower boundary.

Bug 22 — Review rating of 6 is accepted

Endpoint: `POST /api/v1/products/{product_id}/reviews` **Severity:** High

Request:

Method: POST
URL: http://localhost:8080/api/v1/products/1/reviews
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: { "rating": 6, "comment": "Too good" }

Expected: `400 Bad Request` The documentation states: *"A review rating must be between 1 and 5. Anything outside that range must be rejected with a 400 error."* 6 exceeds the maximum.

Actual: `200 OK` The server accepted a rating of 6. Rating validation is not enforced at the upper boundary either.

Bug 23 — Negative review rating is accepted

Endpoint: `POST /api/v1/products/{product_id}/reviews` **Severity:** High

Request:

Method: POST
URL: http://localhost:8080/api/v1/products/1/reviews
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: { "rating": -1, "comment": "Bad" }

Expected: `400 Bad Request` A negative rating is outside the valid 1–5 range and must be rejected.

Actual: `200 OK` The server accepted a negative rating. There is no lower bound check whatsoever on the rating field.

Bug 24 — Review for non-existent product returns 200 instead of 404

Endpoint: `POST /api/v1/products/{product_id}/reviews` **Severity:** High

Request:

Method: POST

URL: http://localhost:8080/api/v1/products/999999/reviews
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: { "rating": 5, "comment": "Good" }

Expected: `404 Not Found` A review cannot be submitted for a product that does not exist. The server must validate the product ID.

Actual: `200 OK` The server accepted the review for product 999999, which does not exist in the database. This creates orphaned review records pointing to non-existent products.

Bug 25 — Average rating is integer-truncated instead of a decimal

Endpoint: `GET /api/v1/products/{product_id}/reviews` **Severity:** Medium

Request:

Method: GET
URL: http://localhost:8080/api/v1/products/1/reviews
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: (none)
[State: two reviews submitted with ratings 3 and 4]

Expected: `average_rating: 3.5` The documentation states: *"The average rating shown must be a proper decimal calculation, not a rounded-down integer."* (3 + 4) / 2 = 3.5.

Actual: `average_rating: 3` The average is returned as the integer 3, truncating the decimal portion. This is an integer division bug in the average calculation.

Bug 26 — Review with missing rating field is accepted

Endpoint: `POST /api/v1/products/{product_id}/reviews` **Severity:** High

Request:

Method: POST
URL: http://localhost:8080/api/v1/products/1/reviews
Headers: X-Roll-Number: 12345
X-User-ID: 25

Body: { "comment": "Nice" }

Expected: `400 Bad Request` `rating` is a required field. A review without a rating must be rejected.

Actual: `200 OK` The server accepted a review with no `rating` field. This creates reviews with null/zero ratings in the database, which will corrupt average calculations.

Bug 27 — Support ticket GET response does not include the `message` field

Endpoint: `GET /api/v1/support/tickets` **Severity:** Medium

Request:

Method: GET
URL: http://localhost:8080/api/v1/support/tickets
Headers: X-Roll-Number: 12345
X-User-ID: 25
Body: (none)

Expected: Each ticket in the list includes `ticket_id`, `subject`, `message`, `status` The documentation states: *"The full message must be saved exactly as written."* It must also be retrievable. The test created a ticket with a specific message, then fetched the list and confirmed the message was stored correctly.

Actual Response for a ticket:

{ "status": "OPEN", "subject": "Order not delivered", "ticket_id": 156 }

The `message` field is entirely absent from the GET response. Users cannot retrieve the messages they submitted.

Summary of Bugs

Bug #	Endpoint	Severity	Type
1	`GET /profile`	Low	Wrong status code (404 vs 400)

2	`PUT /profile`	High	Missing validation (letters in phone accepted)
3	`POST /addresses`	Critical	Valid request rejected
4	`POST /addresses`	High	Boundary violation (5-char street rejected)
5	`POST /addresses`	High	Boundary violation (100-char street rejected)
6	`POST /addresses`	High	Missing validation (5-digit pincode accepted)
7	`POST /addresses`	High	Missing validation (7-digit pincode accepted)
8	`POST /cart/add`	High	Missing validation (quantity 0 accepted)
9	`POST /cart/add`	High	Missing validation (negative quantity accepted)
10	`GET /cart`	Critical	Wrong data (subtotal 104 instead of 360)
11	`GET /cart`	Critical	Wrong data (total -16 instead of correct sum)
12	`POST /cart/add`	Low	Wrong status code (404 vs 400 for missing field)
13	`POST /coupon/apply`	High	Missing validation (below-minimum cart accepted)
14	`POST /coupon/apply`	Critical	Wrong data (fixed discount not applied, stays 0)
15	`POST /checkout`	High	Missing validation (COD over 5000 accepted)
16	`POST /wallet/pay`	High	Wrong data (100.2 deducted instead of 100.0)
17	`POST /orders/{id}/cancel`	Medium	Wrong status code (404 vs 400 for delivered order)

18	`POST /orders/{id}/cancel`	Critical	Wrong behavior (stock decreases instead of restoring)
19	`GET /orders/{id}/invoice`	Medium	Wrong field name (`total_amount` vs `total`)
20	`GET /orders/{id}/invoice`	Medium	Wrong data (total reads as 0 due to field name bug)
21	`POST /products/{id}/reviews`	High	Missing validation (rating 0 accepted)
22	`POST /products/{id}/reviews`	High	Missing validation (rating 6 accepted)
23	`POST /products/{id}/reviews`	High	Missing validation (negative rating accepted)
24	`POST /products/{id}/reviews`	High	Missing validation (non-existent product accepted)
25	`GET /products/{id}/reviews`	Medium	Wrong data (average truncated to integer)
26	`POST /products/{id}/reviews`	High	Missing validation (missing rating field accepted)
27	`GET /support/tickets`	Medium	Missing field (`message` absent from response)